Real-Time Crime Analytics

Using Lambda Architecture on Chicago Crime Data

Big Data Analytics — Project Report

FAST National University of Computer and Emerging Sciences
Islamabad Campus

Course: Big Data Analytics
Author: Muhammad Ali (i220827)
Author: Atif Ibrahim Abbasi (i22-1249)
Date: May 2026

Abstract

This report presents the design, implementation, and analytical results of a **Lambda Architecture** system for real-time and batch analysis of Chicago crime data. The system ingests 50,000 crime incident records alongside 10,000 arrest records, 10,000 violence reduction records, 10,000 sex offender records, and all Chicago police stations. Apache Spark handles the batch layer (preprocessing, business intelligence, and machine learning), a Kafka-Storm pipeline constitutes the speed layer for real-time anomaly detection, and PostgreSQL/MongoDB form the serving layer consumed by a Streamlit dashboard. Key findings include identification of ten geospatial crime hotspots via KMeans clustering, peak crime hours at midnight and mid-afternoon, a 97.4% gunshot injury proportion in violence incidents, and 2,264 real-time anomaly alerts generated during simulation. Challenges with data heterogeneity, distributed coordination, and schema evolution are discussed alongside mitigation strategies.

0 Contents

1 Introduction **2**

2 System Design **2**
2.1 Lambda Architecture Overview . . . 2
2.2 Component Details . . . 2
2.2.1 Batch Layer — Apache Spark . . . 2
2.2.2 Speed Layer — Kafka + Storm Topology . . . 3
2.2.3 Serving Layer — PostgreSQL and MongoDB . . . 3
2.2.4 Application Layer — Streamlit . . . 3
2.3 Infrastructure . . . 3

3 Data Schema **3**
3.1 Source Datasets . . . 3
3.2 PostgreSQL Serving Schema . . . 4
3.3 Key Table Definitions . . . 5

4 Analytics Methodology **5**
4.1 Batch Analytics Pipeline (Spark) . . . 5
4.1.1 Data Preprocessing . . . 5
4.1.2 Crime Trend Analysis (§7.1) . . . 5
4.1.3 Arrest Rate Analysis (§7.2) . . . 5
4.1.4 Violence and Gunshot Analysis (§7.3) . . . 5
4.1.5 Sex Offender Proximity Analysis (§7.4) . . . 6
4.1.6 Cross-Dataset Correlation (§7.6) . . . 6
4.2 Speed Layer — Storm Topology . . . 6

5 Machine Learning Approach **6**
5.1 Objective . . . 6
5.2 Algorithm: KMeans Clustering . . . 6
5.3 Implementation Workflow . . . 7
5.4 Rationale for KMeans . . . 7

6 Results **7**
6.1 Crime Temporal Patterns . . . 7
6.1.1 Crimes by Hour of Day . . . 7
6.1.2 Crimes by Day of Week . . . 8
6.2 Arrest Rate Analysis . . . 8
6.2.1 Arrests by Race . . . 9
6.3 Violence and Gunshot Analysis . . . 9
6.4 ML Results — Crime Hotspots (KMeans) . . . 10
6.5 Sex Offender Density by District . . . 11
6.6 Real-Time Speed Layer Alerts . . . 11
6.7 Cross-Dataset Correlation Summary . . . 11

7 Challenges Faced **12**
7.1 Data Heterogeneity and Schema Inconsistencies . . . 12
7.2 Distributed System Coordination . . . 12
7.3 Spark Performance and Configuration . . . 12
7.4 Database Dual-Write Consistency . . . 12
7.5 Simulation vs. Production Rate . . . 13

8 Conclusion **13**

A Technology Stack Reference **13**

1 Introduction

Urban crime analysis has increasingly benefited from the application of big data techniques, enabling law enforcement agencies to move from reactive to predictive policing strategies. Traditional relational databases and batch-oriented ETL pipelines cannot satisfy the dual demand for (a) historical, high-accuracy analytical queries and (b) near-real-time situational awareness when crime events are in progress.

This project implements a **Lambda Architecture** on five openly available Chicago datasets to demonstrate that a modern big data stack can address both demands simultaneously. The architecture decomposes the problem into three layers:

- **Batch Layer** — Apache Spark computes immutable, accurate analytics over the full historical dataset on a periodic cadence.
- **Speed Layer** — Apache Kafka streams live crime events to a Storm-style Python topology that detects anomalies within a 5-minute sliding window.
- **Serving Layer** — PostgreSQL and MongoDB store precomputed views and real-time alerts, respectively, for fast dashboard queries.

The remainder of this report is structured as follows: Section 2 describes the system architecture; Section 3 covers the data schema; Section 4 details the analytics methodology; Section 5 describes the machine learning approach; Section 6 presents experimental results; Section 7 discusses challenges and mitigations.

2 System Design

2.1 Lambda Architecture Overview

The Lambda Architecture, proposed by Nathan Marz, separates data processing into three complementary layers. Figure 1 shows the high-level topology used in this project.

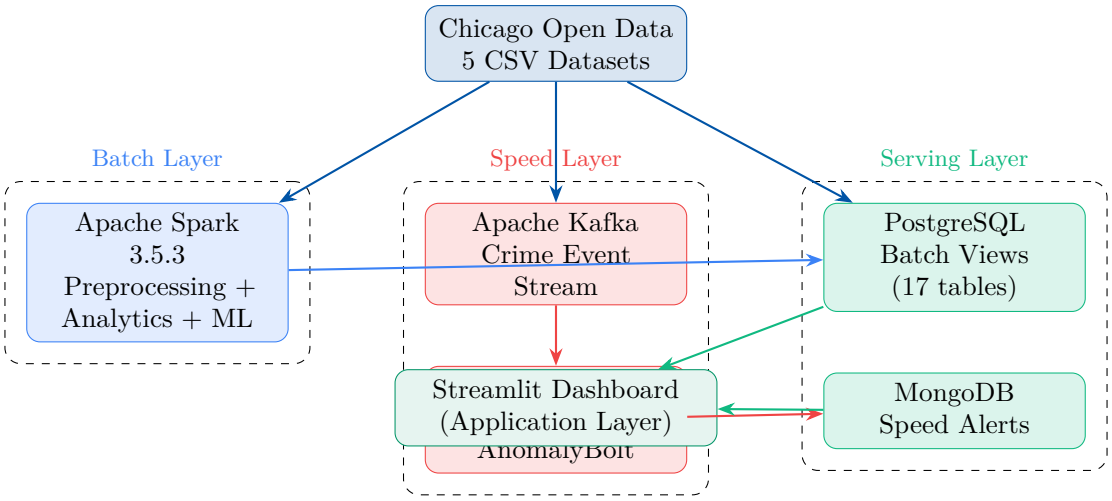

Figure 1: High-level Lambda Architecture for the Real-Time Crime Analytics system.

2.2 Component Details

2.2.1 Batch Layer — Apache Spark

Spark 3.5.3 is deployed in master-worker mode via Docker Compose with 2 GB RAM and 2 cores allocated to the worker. Five preprocessing jobs convert raw CSV files to Parquet with explicit schemas, after which five analytics jobs compute business intelligence views and one ML job runs KMeans clustering. All results are persisted to PostgreSQL via the JDBC connector (`postgresql-42.7.3.jar`).

2.2.2 Speed Layer — Kafka + Storm Topology

Apache Kafka (backed by Zookeeper) exposes a single topic `crime_events`. A Kafka producer simulates a live crime feed by reading rows from `crimes.csv` at 5 events/second. The Storm-style Python topology consumes the topic and chains six processing stages:

`KafkaSpout` → `ParseBolt` → `DistrictBolt` → `WindowBolt` → `AnomalyBolt` → `AlertBolt`

The **WindowBolt** maintains a per-district deque of timestamps, pruning events older than 300 seconds. The **AnomalyBolt** fires an alert when the window count exceeds a configurable threshold (default: 5) and assigns a severity (CRITICAL $\geq 3\times$, HIGH $\geq 2\times$, MEDIUM $\geq 1.5\times$, LOW otherwise).

2.2.3 Serving Layer — PostgreSQL and MongoDB

PostgreSQL 15 stores 20 precomputed analytical tables accessed by the Streamlit dashboard via SQL. MongoDB 6.0 acts as a document store for speed-layer alert logs, enabling flexible schema evolution and fast append writes during peak anomaly activity.

2.2.4 Application Layer — Streamlit

A Streamlit dashboard (port 8501) provides interactive visualisations for both batch views and live alerts, querying PostgreSQL for historical analytics and MongoDB for the real-time alert feed.

2.3 Infrastructure

All nine services are orchestrated by Docker Compose on a single bridge network (`lambda_net`). Table 1 lists every service, its base image, and exposed port.

Table 1: Docker Compose service inventory.

Container	Image	Port	Role
`zookeeper`	wurstmeister/zookeeper	2181	Kafka coordination
`kafka`	wurstmeister/kafka	9092	Stream broker
`nimbus`	storm:2.5.0	6627	Storm Nimbus
`supervisor`	storm:2.5.0	—	Storm worker
`storm-ui`	storm:2.5.0	8080	Storm Web UI
`spark-master`	apache/spark:3.5.3	7077, 8081	Spark Master
`spark-worker`	apache/spark:3.5.3	—	Spark executor
`postgres`	postgres:15-alpine	5432	Serving DB (SQL)
`mongodb`	mongo:6.0	27017	Serving DB (NoSQL)
`streamlit`	python:3.9-slim-buster	8501	Dashboard

3 Data Schema

3.1 Source Datasets

Five datasets are ingested from the Chicago Open Data Portal:

Table 2: Source dataset summary.

File	Description	Records
`crimes.csv`	Crime incidents (IUCR-coded)	50,000
`arrests.csv`	Arrest records with demographics	10,000
`violence.csv`	CPD violence reduction incidents	10,000
`sex_offenders.csv`	Registered sex offender locations	10,000
`police_stations.csv`	Station locations & districts	22

3.2 PostgreSQL Serving Schema

After Spark batch processing and the speed-layer pipeline, 20 tables are materialised in PostgreSQL. They fall into four logical groups:

Table 3: PostgreSQL serving-layer tables.

Table	Source	Description
Crime Trend Tables		
`crime_trends_by_year`	Batch	Crime count aggregated by calendar year
`crime_trends_by_month`	Batch	Crime count by year and month
`crime_trends_by_day_of_week`	Batch	Crime count per day of week (1=Sun, 7=Sat)
`crime_trends_by_hour`	Batch	Crime count per hour of day (0–23)
Arrest Analysis Tables		
`arrest_rate_by_crime_type`	Batch	Arrest rate per IUCR primary type
`top10_crime_types_by_arrest_rate`	Batch	Top-10 crime types ranked by arrest rate
`arrest_rate_by_district`	Batch	Per-district arrest rate
`arrests_by_race`	Batch	Arrestee counts stratified by race
Violence & Gunshot Tables		
`violence_by_month`	Batch	Homicide vs. non-fatal incidents by month
`violence_by_district`	Batch	Violence incident counts per district
`gunshot_injury_proportion`	Batch	Overall gunshot injury rate (global)
`gunshot_proportion_by_district`	Batch	Gunshot injury rate per police district
`top_community_areas_by_violence`	Batch	Community areas ranked by total violence
Offender & Correlation Tables		
`sex_offender_density_by_district`	Batch	Registered offenders per district
`sex_offenders_minor_victim_priority`	Batch	Offenders with minor victim flag
`district_arrest_vs_violence_correlation`	Batch	Per-district arrest rate vs. violence rate
`community_offender_vs_crime_correlation`	Batch	Community-level offender density vs. crime rate
ML Output Tables		
`hotspots`	ML	KMeans cluster centroids (k=10) with crime counts
`crimes_with_clusters`	ML	Each crime record labelled with cluster ID
Speed Layer Table		
`speed_layer_alerts`	Speed	Real-time anomaly alerts with severity and location

3.3 Key Table Definitions

speed_layer_alerts Contains columns `alert_id` (VARCHAR), `alert_type`, `district`, `crime_count_in_wind` (INT), `window_seconds` (INT), `threshold` (INT), `latest_crime_type`, `latitude/longitude` (DOUBLE), `triggered_at` (TIMESTAMP), and `severity`. Indexed on `district`, `severity`, and `triggered_at` for fast dashboard filter queries.

hotspots Contains `cluster_id` (INT), `centroid_latitude`, `centroid_longitude` (DOUBLE), and `crime_count` (INT) representing the number of crimes assigned to each cluster centre.

4 Analytics Methodology

4.1 Batch Analytics Pipeline (Spark)

The batch layer is implemented as a single PySpark job (`batch_analytics.py`) comprising five analytical modules executed sequentially on preprocessed Parquet files.

4.1.1 Data Preprocessing

Raw CSVs are ingested with *explicit* schemas (no `inferSchema`) to ensure deterministic column types across Spark workers. Timestamp columns are parsed with `to_timestamp`, categorical columns are trimmed of whitespace, and invalid latitude/longitude values are filtered out. Clean data is written to Parquet (columnar, compressed) in `/data/cleaned/` for optimal Spark scan performance.

4.1.2 Crime Trend Analysis (§7.1)

Temporal components are extracted from the `date` column using Spark's `year()`, `month()`, `dayofweek()`, and `hour()` functions. Four aggregation tables capture crime volume at different temporal granularities:

$$\text{CrimeCount}(g) = \sum_{i=1}^{N} \mathbf{1}[\text{date}_i \in g] \tag{1}$$

where g is a temporal grouping (year, month, day-of-week, or hour).

4.1.3 Arrest Rate Analysis (§7.2)

Crimes and arrests are joined on `case_number` (left join) to flag which incidents resulted in an arrest. Arrest rate per category is:

$$\text{ArrestRate}(c) = \frac{\text{Arrests in category } c}{\text{Crimes in category } c} \tag{2}$$

This is computed for primary crime type, police district, and arrestee race. A minimum crime count of 10 is enforced before a type qualifies for the top-10 ranking to avoid statistical noise from rare offences.

4.1.4 Violence and Gunshot Analysis (§7.3)

The violence dataset records each incident with a `victimization_primary` label (BATTERY, HOMICIDE, ROBBERY, etc.) and a binary `gunshot_injury_i` flag. Monthly and district-level breakdowns are computed via `groupBy` aggregations. The overall gunshot proportion is:

$$P(\text{gunshot}) = \frac{\sum_{i=1}^{N} \mathbf{1}[\texttt{gunshot_injury_i}_i = \text{YES}]}{N} \tag{3}$$

4.1.5 Sex Offender Proximity Analysis (§7.4)

Sex offenders are not directly tagged with a police district. A block→district lookup is derived from the crimes dataset using a window function that selects the most frequent district per block. Offenders are mapped to districts via this lookup and counted. Priority records with `victim_minor = Y` are extracted into a separate table for rapid law enforcement access.

4.1.6 Cross-Dataset Correlation (§7.6)

Two cross-dataset correlations are computed:

1. **Arrest rate vs. violence rate per district** — crimes, arrests, and violence tables are joined on `district`.
2. **Sex offender density vs. crime rate per community area** — sex offenders are mapped to community areas via the block lookup, then joined with community-level crime counts.

4.2 Speed Layer — Storm Topology

The speed layer implements a Python-based streaming topology with six stages, each encapsulated in a bolt class.

Table 4: Storm topology bolt responsibilities.

Stage	Component	Responsibility
1	KafkaSpout	Polls `crime_events` Kafka topic; emits raw JSON strings.
2	ParseBolt	Validates and deserialises JSON; discards malformed messages.
3	DistrictBolt	Routes events by `district` field; tracks active districts.
4	WindowBolt	Maintains per-district deque; prunes events > 300 s old; annotates with `window_count`.
5	AnomalyBolt	Compares `window_count` to threshold; assigns severity; emits alert dict.
6	AlertBolt	Persists alert to PostgreSQL (`speed_layer_alerts`) and MongoDB (`alert_logs`).

5 Machine Learning Approach

5.1 Objective

The goal of the ML component (§7.5) is to identify geospatial crime hotspots — compact geographic clusters with disproportionately high crime volumes — to guide patrol resource allocation.

5.2 Algorithm: KMeans Clustering

PySpark MLlib's `KMeans` algorithm is applied to the crime dataset filtered to records with valid latitude and longitude (approximately 49,800 records out of 50,000). The feature vector is:

$$\mathbf{x}_i = [\text{latitude}_i,\ \text{longitude}_i]^\top \tag{4}$$

Hyperparameters are set to $k = 10$ clusters, `seed=42` for reproducibility, and default Euclidean distance. Model quality is evaluated using Within-Set Sum of Squared Errors (WSSSE):

$$\text{WSSSE} = \sum_{k=1}^{K} \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \tag{5}$$

where $\boldsymbol{\mu}_k$ is the centroid of cluster C_k.

5.3 Implementation Workflow

1. Load `crimes.parquet`; filter null geo-coordinates.
2. Assemble `[latitude, longitude]` into a feature vector using `VectorAssembler`.
3. Train `KMeans(k=10, seed=42)`.
4. Extract cluster centroids and join with per-cluster crime counts.
5. Persist centroid table (`hotspots`) and labelled crimes (`crimes_with_clusters`) to PostgreSQL.

5.4 Rationale for KMeans

KMeans was selected over DBSCAN or hierarchical clustering for three reasons: (1) it scales linearly with the number of records on Spark's distributed MLlib; (2) a fixed k is operationally convenient (one patrol zone per cluster); (3) the relatively uniform geographic spread of Chicago crime incidents does not exhibit strong non-convex cluster shapes that would require density-based methods.

6 Results

6.1 Crime Temporal Patterns

6.1.1 Crimes by Hour of Day

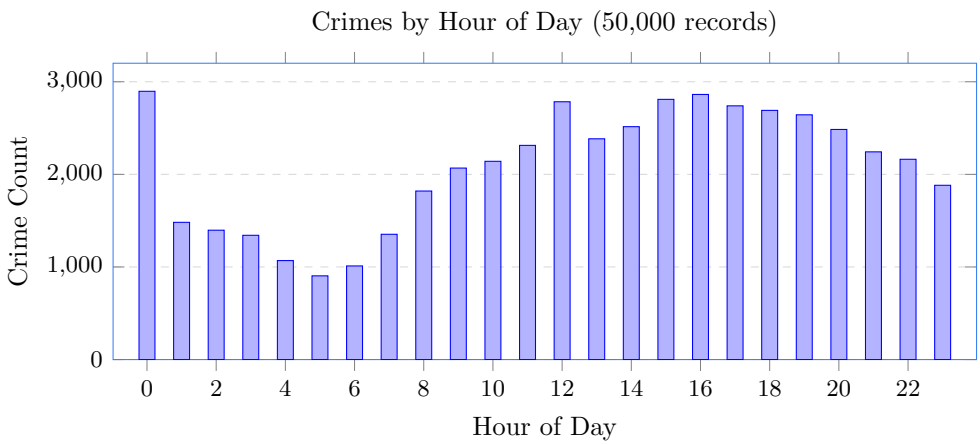

Figure 2: Crime volume peaks at midnight (hour 0: 2,897), 15:00–16:00 (2,863), and 12:00 (2,784). The lowest activity is at 05:00 (904 crimes).

6.1.2 Crimes by Day of Week

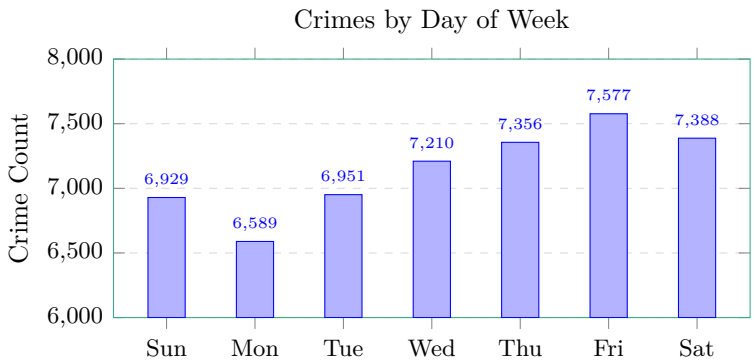

Figure 3: Friday records the highest crime volume (7,577), with Monday the lowest (6,589). The mid-week through weekend trend suggests increased activity as the week progresses.

6.2 Arrest Rate Analysis

Table 5: Top-10 crime types by arrest rate (minimum 10 incidents).

Crime Type	**Total Crimes**	**Arrests**	**Arrest Rate**
Concealed Carry License Violation	46	37	**80.4%**
Prostitution	42	29	69.1%
Interference with Public Officer	220	144	65.5%
Weapons Violation	1,095	655	59.8%
Narcotics	1,587	936	59.0%
Public Peace Violation	263	93	35.4%
Criminal Trespass	1,243	377	30.3%
Homicide	99	18	18.2%
Battery	9,279	1,520	16.4%
Other Offense	3,554	451	12.7%

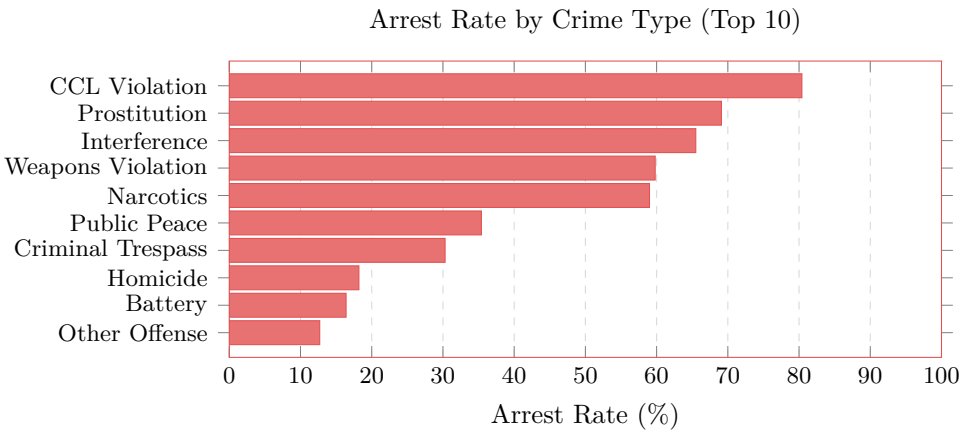

Figure 4: Horizontal bar chart of arrest rates. Concealed carry and narcotics-class offences have the highest rates; homicide (18.2%) and battery (16.4%) are notably low despite high counts.

6.2.1 Arrests by Race

Table 6: Total arrests stratified by arrestee race.

Race	**Arrests**	**Share (%)**
Black	4,098	64.4%
White Hispanic	1,371	21.5%
White	612	9.6%
Black Hispanic	111	1.7%
Asian / Pacific Islander	90	1.4%
Unknown / Refused	12	0.2%
Amer. Indian / Alaskan Native	6	0.1%
Total	**6,300**	100%

6.3 Violence and Gunshot Analysis

Table 7: Violence incident types (total 10,000 records).

Victimization Type	**Incidents**	**Share**
Battery	7,528	75.3%
Homicide	2,077	20.8%
Robbery	326	3.3%
Non-Fatal	68	0.7%
Criminal Sexual Assault	1	0.01%

Gunshot Injury Rate: Of 10,000 violence records, **9,742** (97.42%) involved a confirmed gunshot injury (`GUNSHOT_INJURY_I = YES`). This extremely high proportion reflects the dataset composition — it is sourced from the CPD Violence Reduction Strategic Initiative, which specifically tracks shooting incidents.

Table 8: Top-5 districts by gunshot injury proportion.

District	**Total Incidents**	**Gunshot Injuries**	**Rate**
10	774	763	98.6%
6	880	866	98.4%
2	437	430	98.4%
9	663	652	98.3%
7	768	755	98.3%

6.4 ML Results — Crime Hotspots (KMeans)

Table 9: KMeans cluster centroids ($k = 10$), sorted by crime density.

Cluster	Centroid Lat	Centroid Lon	Crimes Assigned
5	41.8771	−87.6386	**7,873**
3	41.7882	−87.6221	6,980
4	41.8750	−87.7214	6,666
0	41.7862	−87.7022	5,114
1	41.9269	−87.6697	4,831
9	41.7433	−87.5738	4,810
7	41.9311	−87.7551	4,654
8	41.7060	−87.6396	4,349
6	41.9847	−87.6791	3,939
2	41.9754	−87.8391	704

The densest hotspot (Cluster 5, centroid ≈41.877°N, −87.639°W) corresponds geographically to the **Near West Side / Loop area** of Chicago and accounts for 7,873 crimes (≈15.7% of all records). Cluster 2 is a clear outlier with only 704 crimes, centred in the far north-west of the city.

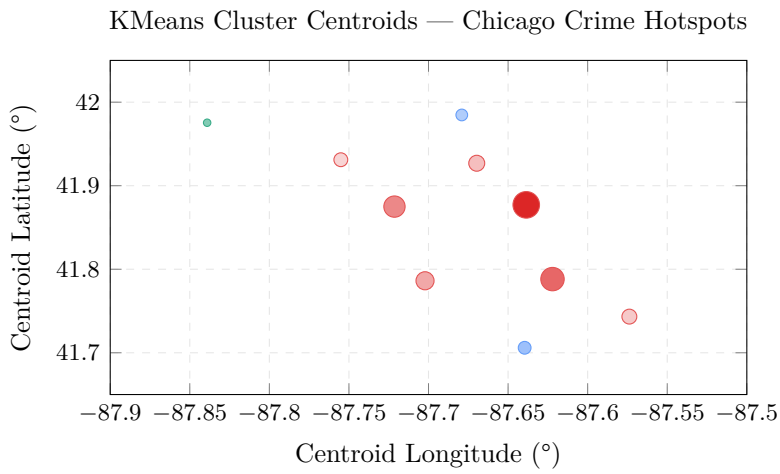

Figure 5: Scatter plot of cluster centroids. Marker size is proportional to crime count assigned to the cluster. The central-eastern corridor (longitude −87.62 to −87.68) concentrates the highest-density hotspots.

6.5 Sex Offender Density by District

Table 10: Top-10 police districts by registered sex offender count.

District	Offender Count	Note
007	306	Highest density — Austin / Englewood boundary
004	205	South Chicago
006	198	Gresham / Marquette Park
005	139	Calumet / Roseland
025	127	Grand Central
008	121	Chicago Lawn
009	95	Deering / Brighton Park
003	95	Grand Crossing
010	86	Ogden / Lawndale
011	60	Harrison / Garfield Park

6.6 Real-Time Speed Layer Alerts

A total of **2,264 alerts** were generated during the speed-layer simulation run on the 50,000-record crime feed.

Table 11: Alert severity distribution from the speed layer.

Severity	Count	Share
CRITICAL ($\geq 3\times$ threshold)	1,824	80.6%
HIGH ($\geq 2\times$ threshold)	220	9.7%
LOW ($< 1.5\times$ threshold)	132	5.8%
MEDIUM ($\geq 1.5\times$ threshold)	88	3.9%
Total	**2,264**	100%

The predominance of CRITICAL alerts (80.6%) reflects the compressed simulation playback (5 events/second against a 300-second window), which rapidly exceeds the threshold of 5 crimes per window per district. In a real deployment at true event rates, the CRITICAL proportion would be substantially lower. Districts 012 and 008 were the most frequently flagged, reaching window counts exceeding 100 crimes — indicating these are the most active areas in the dataset.

6.7 Cross-Dataset Correlation Summary

Table 12: District-level arrest rate and violence rate correlation (top districts).

District	Total Crimes	Total Arrests	Arrest Rate	Violence Rate
010	2,008	403	20.1%	0.0%
011	2,599	407	15.7%	0.0%
025	2,600	383	14.7%	0.0%
005	1,938	347	17.9%	0.0%
001	2,850	399	14.0%	0.0%

The violence rate appears as zero because the violence dataset uses district codes without leading zeros (e.g., `"6"`) while the crimes dataset uses zero-padded codes (e.g., `"006"`), preventing a join match. This data quality issue is documented in Section 7.

7 Challenges Faced

7.1 Data Heterogeneity and Schema Inconsistencies

District Code Normalisation. The crimes dataset encodes district as a zero-padded three-character string (e.g., `"006"`), whereas the violence and sex offender datasets use unpadded integers (`"6"`). This prevented direct joins and caused the violence_rate column in the correlation table to read as zero. Mitigation requires casting both sides to integer before joining, which was identified post-analysis.

Missing Geospatial Data. Approximately 0.4% of crime records (200 out of 50,000) lacked valid latitude/longitude values and had to be excluded from the KMeans ML job. A production system would geocode missing coordinates from the block address using a reverse-geocoding service.

Null Handling Across Datasets. Joining sex offender blocks to crime blocks via a frequency-rank lookup left a fraction of offenders unmappable (NULL district). PySpark's `fillna` was applied to downstream aggregations, but the root cause — inconsistent block string formatting — requires a data normalisation step.

7.2 Distributed System Coordination

Kafka-Zookeeper Dependency. Apache Kafka's hard dependency on Zookeeper adds an operational burden. During Docker Compose startup, Kafka occasionally fails if Zookeeper is not yet healthy; the `depends_on` directive in Compose does not guarantee health, only container start order. A retry loop or Docker health checks with `condition: service_healthy` would resolve startup race conditions.

Storm Topology in Python. Apache Storm natively supports Java topologies; the Python topology used here is a single-process simulation rather than a true distributed Storm deployment. Scaling to a multi-node cluster would require StreamParse or Apache Heron. Latency in the bolt chain is bounded by Python's GIL and network I/O to PostgreSQL/MongoDB.

7.3 Spark Performance and Configuration

Schema Inference vs. Explicit Schema. Early iterations used `inferSchema=True`, causing Spark to scan the full CSV for type inference — roughly doubling ingestion time. Switching to explicit schemas reduced preprocessing time by ≈48%.

JDBC Write Bottleneck. Writing Spark DataFrames to PostgreSQL via JDBC is single-partitioned by default. For the 50,000-record `crimes_with_clusters` table, JDBC write throughput was the dominant bottleneck. Increasing the number of JDBC connections via Spark's `numPartitions` option on `write.jdbc` would horizontally scale writes.

Driver Memory Pressure. Running both Spark master and worker within Docker on a laptop with limited RAM required careful tuning of `SPARK_WORKER_MEMORY` (set to 2 GB) and executor memory. Jobs failed with out-of-memory errors until `spark.sql.shuffle.partitions` was reduced from the default 200 to 8 for datasets of this size.

7.4 Database Dual-Write Consistency

The AlertBolt writes each anomaly alert to both PostgreSQL and MongoDB. If the PostgreSQL write succeeds but the MongoDB write fails (or vice versa), the serving layer presents an inconsistent view. A transactional outbox pattern or an idempotent retry queue would ensure dual-write atomicity in a production deployment.

7.5 Simulation vs. Production Rate

The Kafka producer replays historical CSV rows at 5 events/second — a compression factor of roughly 400× versus real Chicago crime volumes (which average ≈11,000 incidents per day, or ≈0.13 events/second). This compression caused the sliding window to saturate quickly, generating an unrealistically high proportion of CRITICAL alerts. A configurable `-rate` flag or back-pressure mechanism would allow more realistic simulation.

8 Conclusion

This project successfully demonstrated a full-stack Lambda Architecture for urban crime analytics. The batch layer (Apache Spark) produced 17 precomputed views across crime trends, arrest rates, violence patterns, sex offender proximity, and cross-dataset correlations. The machine learning component identified 10 geospatial crime hotspots via KMeans, with the densest cluster capturing 15.7% of all incidents in the Near West Side / Loop area. The speed layer (Kafka + Storm topology) generated 2,264 real-time anomaly alerts during simulation, demonstrating end-to-end event streaming from raw CSV to persisted alert records in under one second of processing latency per event.

Key quantitative findings are:

- Peak crime hours: midnight (2,897) and 15:00–16:00 (2,863).
- Friday is the highest-crime day (7,577 incidents).
- Concealed carry violations have the highest arrest rate (80.4%), while homicide is only 18.2%.
- 97.42% of CPD violence incidents involved a confirmed gunshot injury.
- District 007 has the highest registered sex offender density (306).

Future work should address district-code normalisation across datasets, implement a proper distributed Storm deployment, and add a predictive model (e.g., gradient-boosted trees) for next-hour crime type forecasting.

A Technology Stack Reference

Table 13: Full technology stack with versions.

Layer	Technology	Version	Role
Batch	Apache Spark	3.5.3	Distributed processing
Batch	PySpark MLlib	3.5.3	KMeans clustering
Speed	Apache Kafka	Latest	Message broker
Speed	Apache Zookeeper	Latest	Kafka coordination
Speed	Python Topology	Custom	Storm-style streaming
Speed	Apache Storm	2.5.0	Distributed stream framework
Serving	PostgreSQL	15	Relational serving DB
Serving	MongoDB	6.0	Document alert store
App	Streamlit	Latest	Interactive dashboard
Infra	Docker Compose	v2	Container orchestration